



**合肥工业大学
计算机与信息学院**

**计算机科学与技术专业新生研讨课
课程报告**

姓 名：杜晓川

专业班级： 计算机科学与技术 2023-3 班

学 号： 2023212245

2023 年 12 月 25 日

目录

1. 初识 Python.....	4
1.1 什么是 Python.....	4
1.2 Python 的优点.....	5
2. 安装 Python 和 PyCharm.....	6
2.1 为什么安装 Python 和 PyCharm.....	6
2.2 系统要求.....	6
2.3 安装 Python.....	6
2.4 验证是否安装成功 Python.....	8
2.5 安装 PyCharm.....	9
3. Python 基础语法.....	11
3.1 创建一个 Python 文件.....	11
3.2 “print” 的使用.....	12
3.3 变量.....	15
3.4 数学运算.....	16
3.5 注释.....	17
3.6 数据类型.....	17
3.7 input（）函数.....	21
3.8 if 条件语句.....	22
3.9 for 循环.....	24
3.10 while 循环.....	25
3.11 格式化字符串.....	26
3.12 函数.....	27
3.13 引入函数库.....	27
4 面向对象编程.....	30
4.1 类.....	30
4.2 继承.....	30
5.程序处理.....	31
5.1 异常处理.....	31
5.2 程序测试.....	33
6.写在最后.....	35

6.1 心得体会	35
6.2 参考材料	35

1. 初识 Python

1.1 什么是 Python



一、Python 的历史

Python 是一门优雅而健壮的编程语言，它继承了传统编译语言的强大性和通用性，同时也借鉴了脚本语言和解释语言的易用性。

Python 是由创始人贵铎·范·罗萨姆（Guido van Rossum）在阿姆斯特丹于 1989 年圣诞节期间，为了打发圣诞节的无趣，开发的一个新的解释型脚本语言。之所以选中 Python（大蟒蛇的意思）作为该编程语言的名字，是因为他是 BBC 当时正在热播的喜剧连续剧“Monty Python”的爱好者。

Python 的第一个公开发行人版发行于 1991 年，它是纯粹的自由软件，源代码和解释器（CPython）都遵循 GPL (GNU General Public License) 协议。

二、Python 的设计思想

Python 被设计成是“符合大脑思维习惯”的，采用极简主义的设计理念，加以统一规范的交互模式。这使得 Python 易于学习、理解和记忆。Python 开发者的哲学是“用一种方法，最好是只有一种方法来做一件事”。

Python 是完全面向对象的编程语言，函数、模块、数字、字符串等内置类型都是对象。它的类支持多态、操作符重载、和多重继承等高级 OOP 概念，并且 Python 特有的简洁的语法和类型使得 OOP 十分易于使用。当然 OOP 只是 Python 的一个选择而已，就像 C++ 一样，Python 既支持面向对象编程，也支持面向过程编程的模式。

Python 是一种解释型语言，目前 Python 的标准实现方式是将源代码的语句编译（转换）为字节码格式，然后通过解释器将字节码解释出来。Python 没有将代码编译成底层的二进制代码，所以相较于 C 和 C++ 等编译型语言，Python 的执行速度会慢一些。但是 Python 的解释型语言特性提高了开发者开发速度，同时使它拥有解释型语言易于编写和调试等优点。

Python 本身被设计为可扩展的，并非所有的特性和功能都集成到语言核心。Python 提供了丰富的 API 和工具，以便程序员能够轻松地使用 C/C++ 语言来编写扩充模块。Python 为我们提供了非常完善的基础代码库，覆盖了正则表达式、网络、多线程、GUI、数据库、等领域。除了内置的库外，Python 还有大量的第三方库，供你直接使用。

Python 编译器本身也可以被集成到其它需要脚本语言的程序内，因此，很多人还把 Python

作为一种“胶水语言”（glue language）使用，使用 Python 将其他语言编写的程序进行集成和封装。2004 年，Python 已在 Google 内部使用。Google Engine 使用 C++编写性能要求极高的部分，然后用 Python 或 Java/Go 调用相应的模块。他们的目的是 “Python where we can, C++ where we must”，在操控硬件的场合使用 C++，在快速开发时候使用 Python。

1.2 Python 的优点

1、软件质量高

Python 秉承了简洁、清晰的语法，以及高度一致的编程模式。始终如一的设计风格，可以保证开发出相当规范的代码。

针对错误，Python 提供了“安全合理”的退出机制。Python 支持异常处理，能有效捕获和处理程序运行时发生的错误，使你能够监控这些错误并进行处理。

Python 代码能打包成模块和包，方便管理和发布，很适合团队协作开发。

2、开发速度快

Python 致力于开发速度的最优化：简洁的语法、动态的类型、无需编译、丰富的库支持等特性使得程序员可以快速的进行项目开发。Python 往往只要几十行代码就可以开发出需要几百行 C 代码的功能。

Python 解析器能很方便地进行代码调试和测试，也可作为一个编程接口嵌入一个应用程序中。这就使的在开发过程中可以直接进行调试，而避免了耗时而又麻烦的编译过程，大大提高了开发的速度和效率。

在 Python 中，由于内存管理是由 Python 解释器负责的，所以开发人员就可以从内存管理事务中解放出来，仅仅致力于开发计划中首要的应用程序设计。这使得 Python 编写的程序错误更少、更加健壮、开发周期更短。

3、功能强大

Python 的功能足够强大，本身也足够强壮，它还有许多面向其他系统的接口，所以完全可以使用 Python 开发整个系统的原型。

为了完成更多特定的任务，Python 内置了许多预编码的库工具，从正则表达式到网络编程，再到数据库编程都支持。在 web 领域、数据分析领域等，Python 还有强大的框架帮你快速开发你的服务。例如：Django、TruboGears、Pylons 等。

4、易于扩展

Python 易于扩展，（对于 CPython）可以通过 C 或 C++编写的模块进行功能扩展，使其能够成为一种灵活的黏合语言，可以脚本化处理其他系统和组件的行为。

5、跨平台

Python 是跨平台的。在各种不同的操作系统上（Linux、windows、MacOS、Unix 等）都可以看到 Python 的身影。因为 Python 是用 C 写的，又由于 C 的可移植性，使得 Python 可以运行在任何带有 ANSI C 编译器的平台上。尽管有一些针对不同平台开发的特有模块，但是在任何一个平台上用 Python 开发的通用软件都可以稍事修改或者原封不动的在其他平台上运行。这种可移植性既适用于不同的架构，也适用于不同的操作系统

2. 安装 Python 和 PyCharm

2.1 为什么安装 Python 和 PyCharm

安装 Python 即指安装 Python 解释器，只有安装了这个应用电脑才能读懂我们的 Python 程序。安装了 Python 后我们自然还需要一个能写 Python 代码的软件，即 Python 编译器。Python 编译器多种多样



这里我们选择的是 PyCharm，这是最适合初学者使用的 Python 编译器。和其他编译器相比，它主要有四个优点：

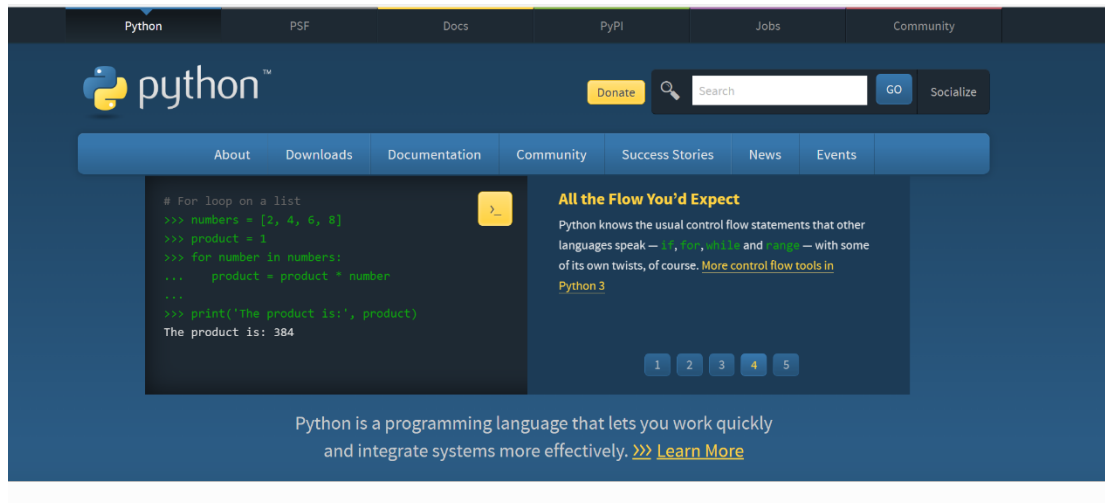
1. 的代码编辑环境
 2. 的智能代码辅助功能（智能代码补全、代码检查、实时错误高亮显示）
 3. 的内建开发者工具（VCS、数据库、远程开发、调试、测试、性能分析...）
 4. 的 Web 开发和数据科学支持（Django、Flask、JS、notebook、pandas...）
- 同时，PyCharm 作为跨平台应用程序，与 Linux、macOS 和 Windows 平台兼容。

2.2 系统要求

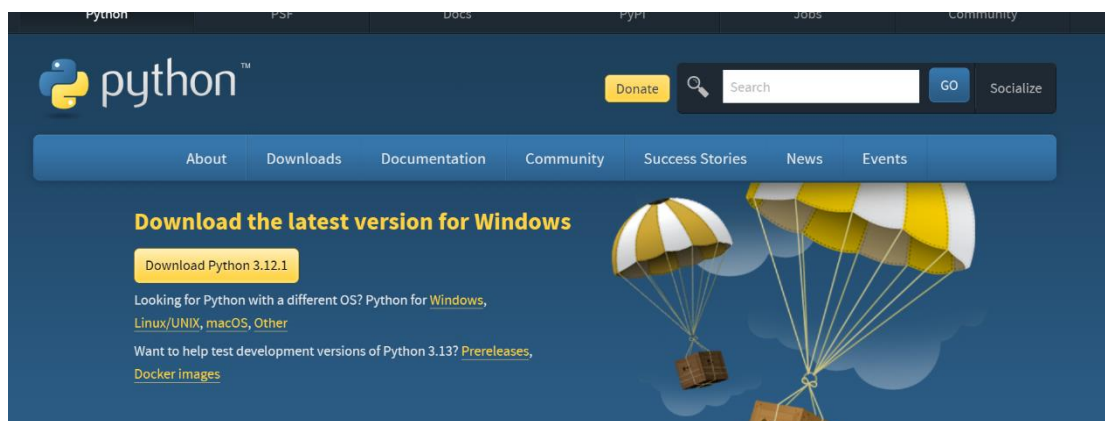
最新版 Python 要求 Windows7 以上

2.3 安装 Python

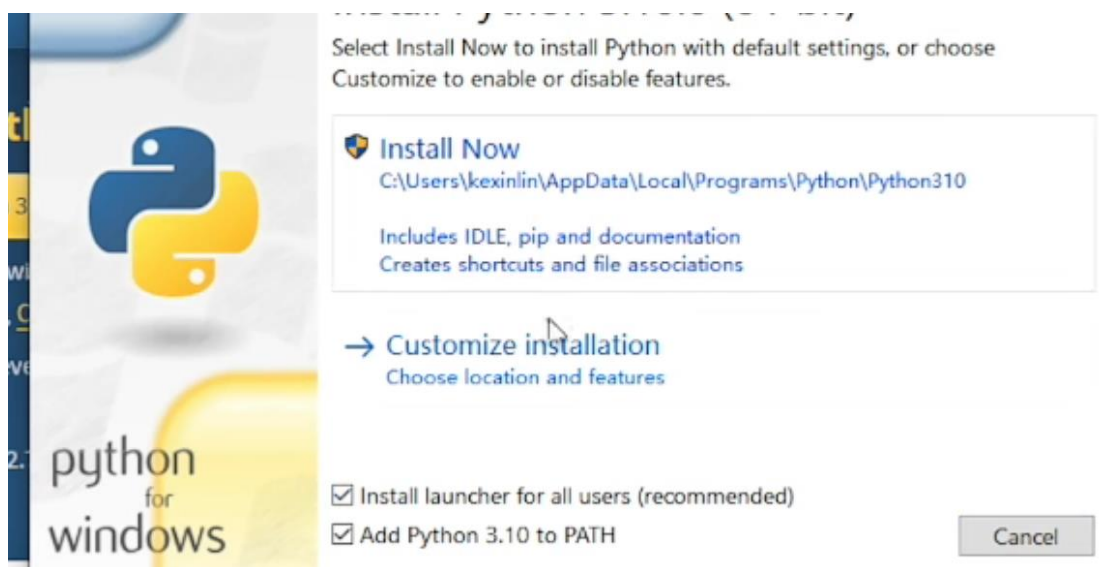
官网：<https://www.python.org/downloads/windows/>



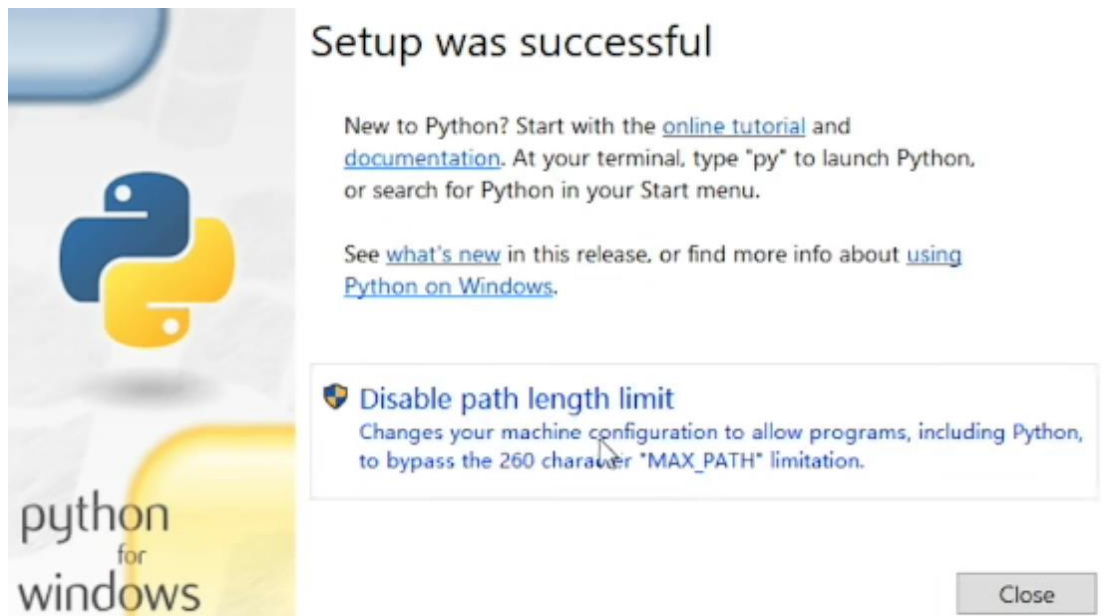
进入官网，点击 Downloads 图标，跳转页面



直接点击黄色按钮下载，下载完毕后安装



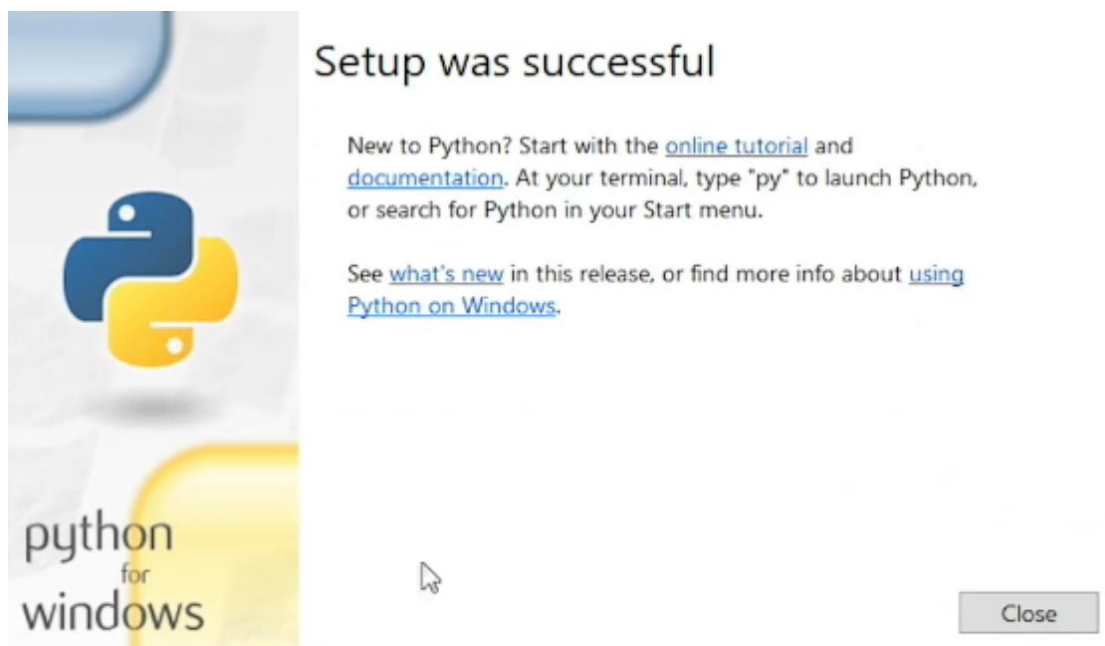
打开文件会弹出这个页面，最下面那个框打上√，点击“Install Now”直接安装



遇到这个页面，点击鼠标所指内容。

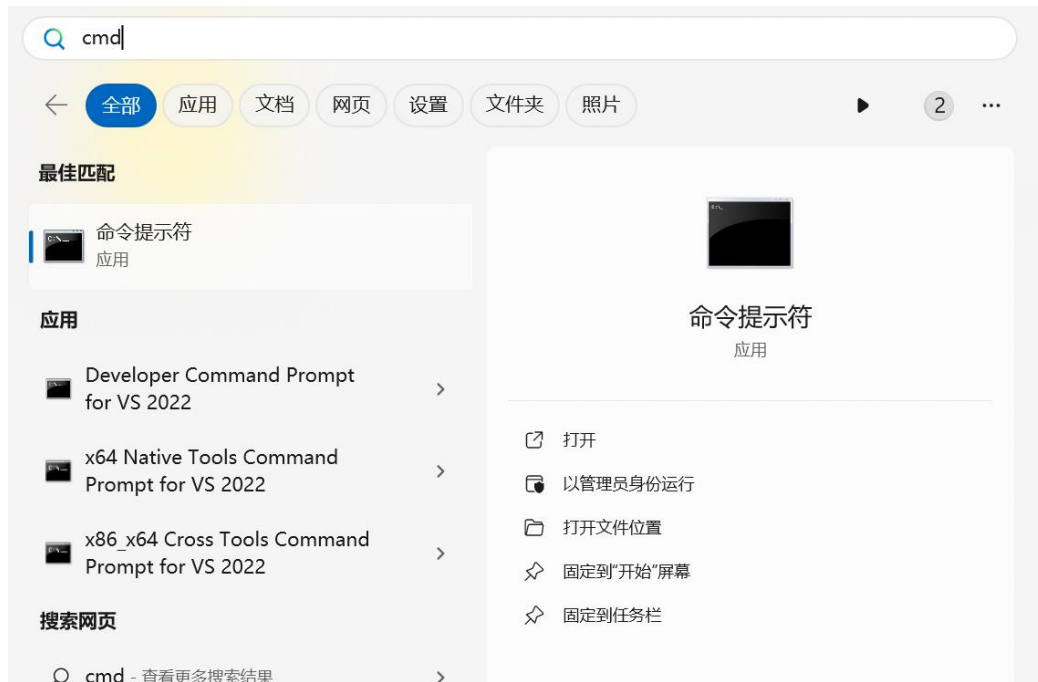
遇到“你要允许此应用对你的设备进行更改吗”弹窗选是。

安装完毕后是这样的：

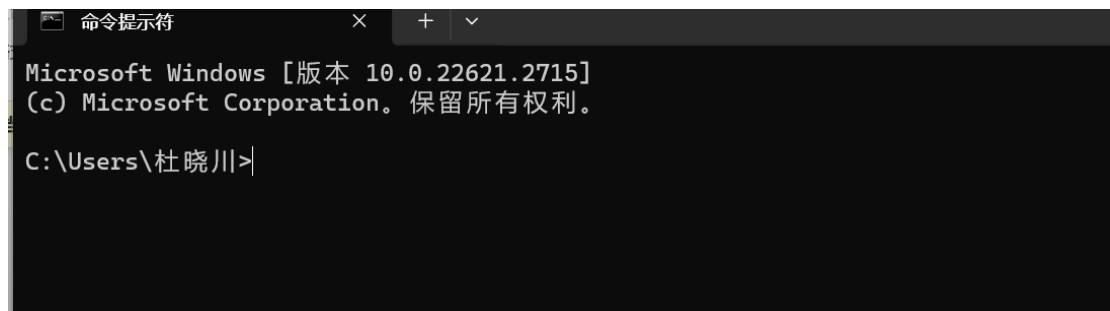


2.4 验证是否安装成功 Python

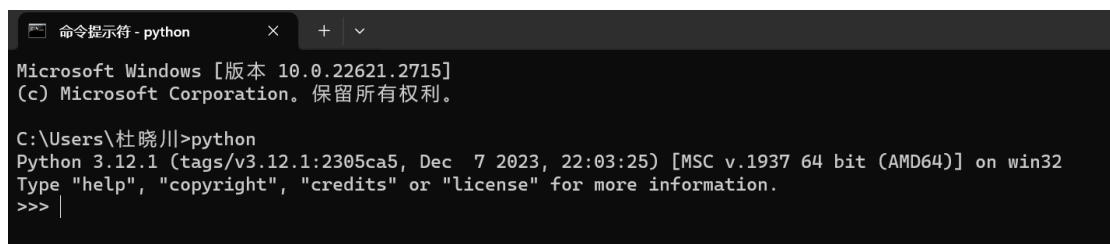
打开电脑搜索窗口输入“cmd”



打开应用命令提示符



输入“Python”后如果显示下面一串可明显看出与 Python 有关的文字说明 Python 安装成功

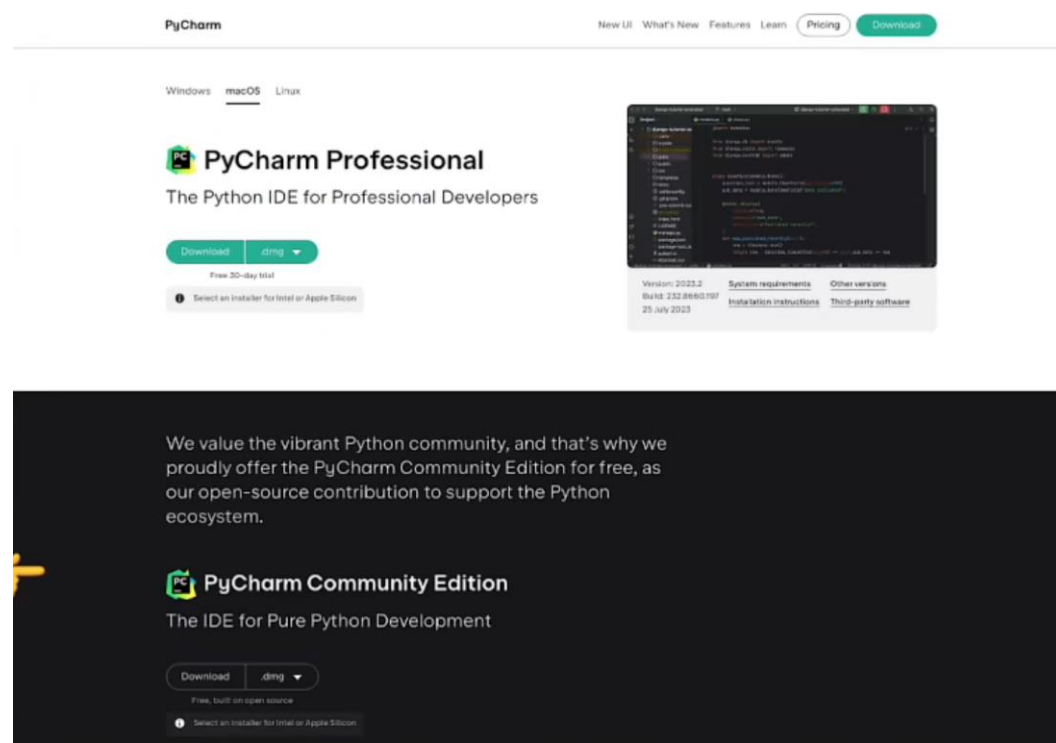


2.5 安装 PyCharm

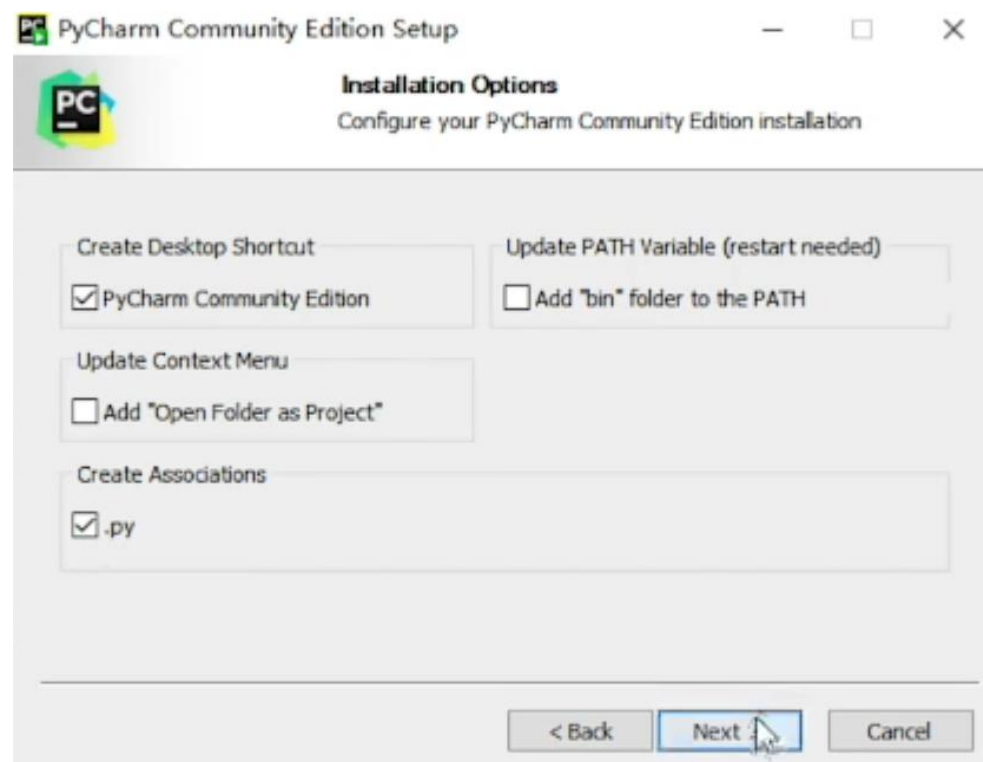
网址：[PyCharm: JetBrains 为专业开发者提供的 Python IDE](#)

PyCharm 有两个版本，上面白色的是专业版，需要收费。选择下载下面黑色的社区版即可，

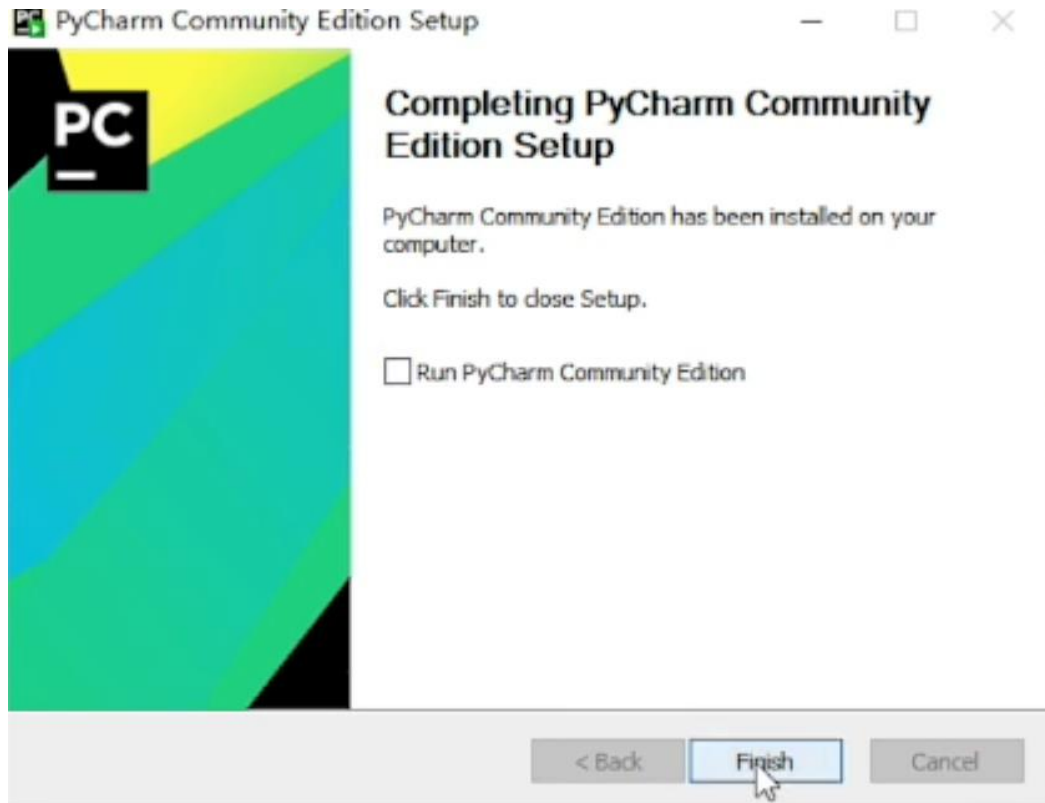
这是免费的



将下面 1、3 图标打√，第一个是创建快捷方式的，第三个是选择所有的 Python 文件都用 PyCharm 打开。



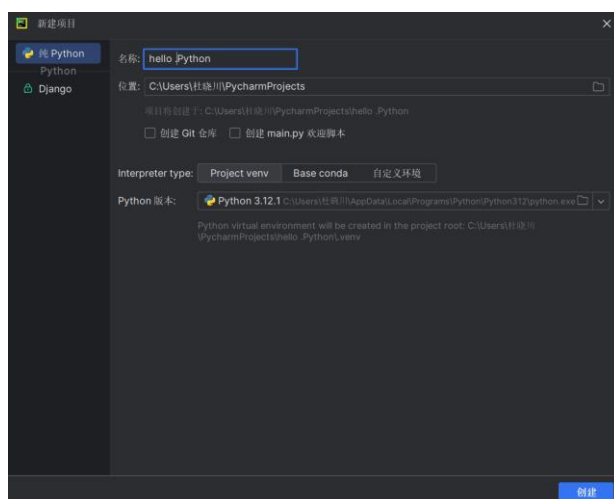
安装成功后是这样的：



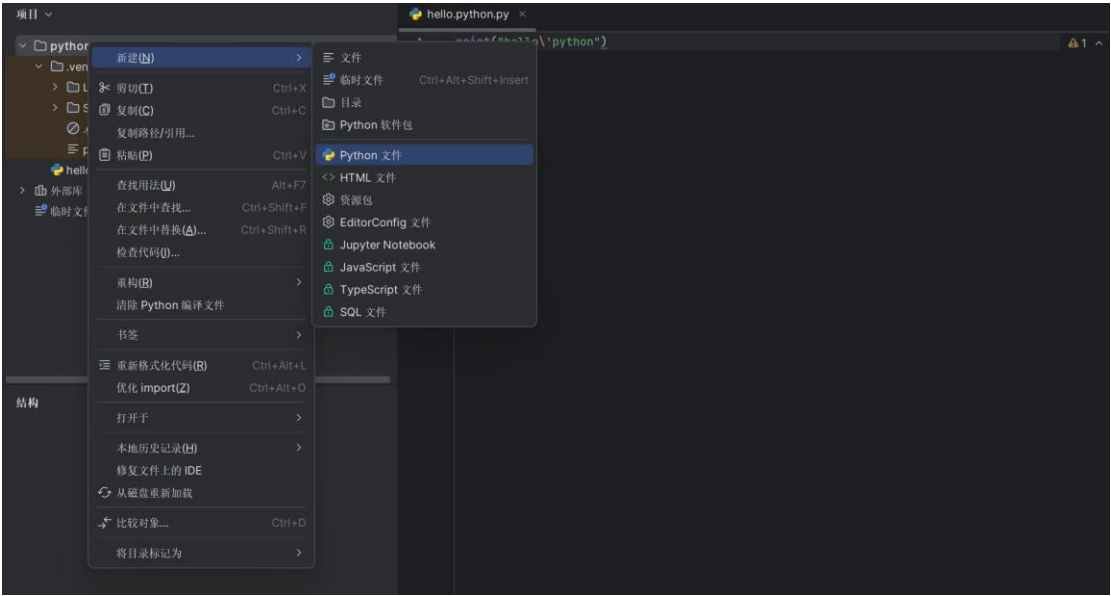
3. Python 基础语法

3.1 创建一个 Python 文件

打开 PyCharm，点击“new project”，可以自己修改名称，文件位置系统会自动给一个，如果这个位置不好找，可以自己找个易于寻找的位置存储，下面的“Python”版本可以选择不同的 Python 解释器，当然前提是你得下载了多种解释器



点击“Python project”选择“新建”创建 Python，然后就可以在你所创建的这个文件里写代码了。



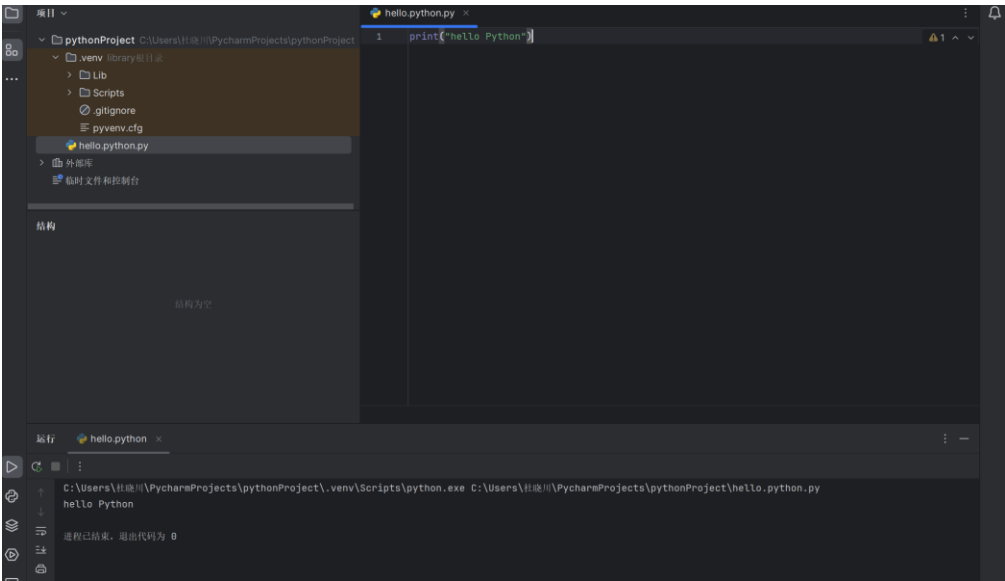
3.2 “print” 的使用

1. print 基本语法

在写代码之前，我们首先要了解 Python 代码的读取运行规则。Python 代码以行为单位，读取一行，运行一行，和 C++用“;”来分隔不同代码是不一样的。当然代码都是用英文符号。

“print”（打印）的用法：`print(“hello Python”)` 这里使用单双引号都可以

然后在点击右键选择运行



2. print 进阶

(1) 字符串连接

我们可以使用 “+” 对字符串进行连接，形如：`print(“hello” + ’ ’ + ” Python” )`

```
1 print("hello"+" "+"Python")
```

```
hello Python
```

进程已结束，退出代码为 0

(2) 单双引号转义

我们可以使用单双引号输出我们需要的内容

如果我们想要输出 `you say :” Python”`

是不可以这样写的：`print(“you say :” Python” ” )`

此时编译器会将 “you say :” 当成一组，运行到后面会直接报错

正确的写法是：`print( ‘you say :” Python” ’ )`

这样，编译器就会以 ‘ ’ 为一组，明白 “ ” 是我们想打出来的内容

```
1 print('you say:"Python"')
```

```
you say:"Python"
```

如果遇到标点很麻烦，单双引号不好用时，我们可以直接用 “\”

反斜杠后面加上标点就代表这是我想输出出来的标点

```
1 print('you say:\"Python\"')
```

```
you say:"Python"
```

进程已结束，退出代码为 0

(3) 换行

如果我们想换行打印，最直接的是使用多个“print”

```
1 print('How are you?')
2 print("I'm fine,thank you.")
```

```
How are you?
I'm fine,thank you.

进程已结束，退出代码为 0
```

但这有点麻烦，和 C++ 一样，使用“\n”可以直接换行

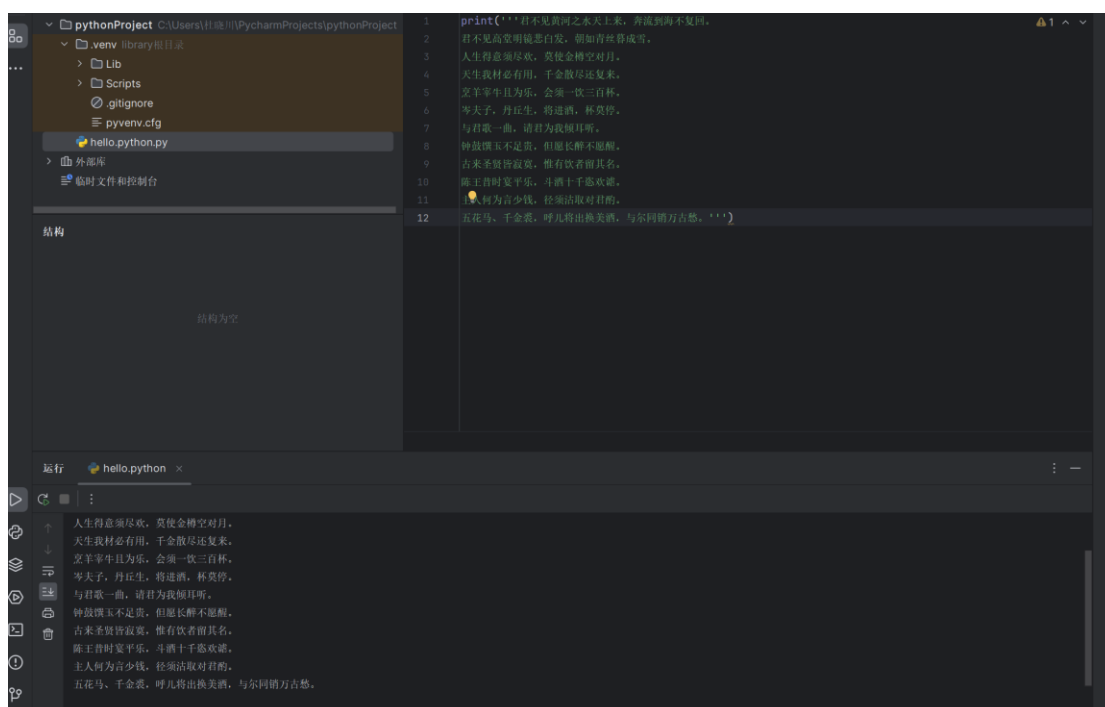
```
1 print("How are you?\nI'm fine thank you.")
```

```
How are you?
I'm fine thank you.

进程已结束，退出代码为 0
```

(4) 三引号跨行字符串

当我们需要多次换行时，使用“\n”会显得代码不清晰美观，同时也比较麻烦，此时我们就可以使用三引号，三引号里的内容如果在代码里是换行的，打印出来同样会换行



```
1 print("""君不见黄河之水天上来，奔流到海不复回。
2 君不见高堂明镜悲白发，朝如青丝暮成雪。
3 人生得意须尽欢，莫使金樽空对月。
4 天生我材必有用，千金散尽还复来。
5 烹羊宰牛且为乐，会须一饮三百杯。
6 岑夫子，丹丘生，将进酒，杯莫停。
7 与君歌一曲，请君为我倾耳听。
8 钟鼓馔玉不足贵，但愿长醉不愿醒。
9 古来圣贤皆寂寞，惟有饮者留其名。
10 陈王昔时宴平乐，斗酒十千恣欢谑。
11 主人何为言少钱，径须沽取对君酌。
12 五花马，千金裘，呼儿将出换美酒，与尔同销万古愁。""")
```

运行结果：

```
君不见黄河之水天上来，奔流到海不复回。
君不见高堂明镜悲白发，朝如青丝暮成雪。
人生得意须尽欢，莫使金樽空对月。
天生我材必有用，千金散尽还复来。
烹羊宰牛且为乐，会须一饮三百杯。
岑夫子，丹丘生，将进酒，杯莫停。
与君歌一曲，请君为我倾耳听。
钟鼓馔玉不足贵，但愿长醉不愿醒。
古来圣贤皆寂寞，惟有饮者留其名。
陈王昔时宴平乐，斗酒十千恣欢谑。
主人何为言少钱，径须沽取对君酌。
五花马，千金裘，呼儿将出换美酒，与尔同销万古愁。
```

3.3 变量

Python 中的变量直接起名取等即可，不需要设置数据类型

```
1 my_age="19";
2 print(my_age)
3 my_age="十九"
4 print(my_age)
```

```
19
十九

进程已结束，退出代码为 0
```

变量的命名规则

在 Python 中，变量的命名需要遵循以下规则：

1. 变量名只能包含字母、数字和下划线：变量名可以由字母（大小写均可）、数字和下划线组成，但不能使用空格或其他特殊字符。
2. 变量名不能以数字开头：变量名可以以字母或下划线开头，但不能以数字开头。
3. 变量名区分大小写。
4. 避免使用 Python 关键字：不能使用 Python 语言中的关键字（如`if`、`else`、`for`等）作为变量名。
5. 推荐使用有意义的名字：变量名应该能够清晰地表达变量所代表的含义，这样可以增加代码的可读性。

大家默认俗成的规则

在实际使用中，Python 社区通常遵循一些命名惯例，以提高代码的可读性和一致性。以下是一些常见的 Python 命名规则和惯例：

1. 蛇形命名法 (Snake Case)：在 Python 中，通常使用蛇形命名法来命名变量，即用小写字母和下划线来分隔单词，例如 `my_variable_name`。
2. 类名使用大写字母开头的驼峰命名法：在定义类名时，通常使用大写字母开头的驼峰命名法，例如 `MyClassName`。
3. 使用有意义的变量名：Python 社区鼓励使用具有描述性的变量名，以提高代码的可读性。例如，使用 `total_students` 而不是 `ts`。
4. 避免使用单个字符作为变量名：除非是计数器或者循环变量，通常应避免使用单个字符作为变量名，因为这样的变量名通常不够具有描述性。
5. 避免使用下划线开头的变量名：通常，以单个下划线开头的变量名被视为“内部使用”或“私有”的变量，而以双下划线开头的变量名则表示类的私有成员。

这些命名规范和惯例有助于提高代码的可读性和一致性，也使得代码更易于维护和协作。虽

然这些规则不是强制性的，但遵循它们通常会使你的代码更符合 Python 社区的习惯。

3.4 数学运算

数字类型

字符串：“6”，不可用于计算。

整数型：6。

浮点型（小数）：6.0。

运算符

加减乘除和 C++ 一样，分别是 “+” “-” “*” “/”。

次方表示：**

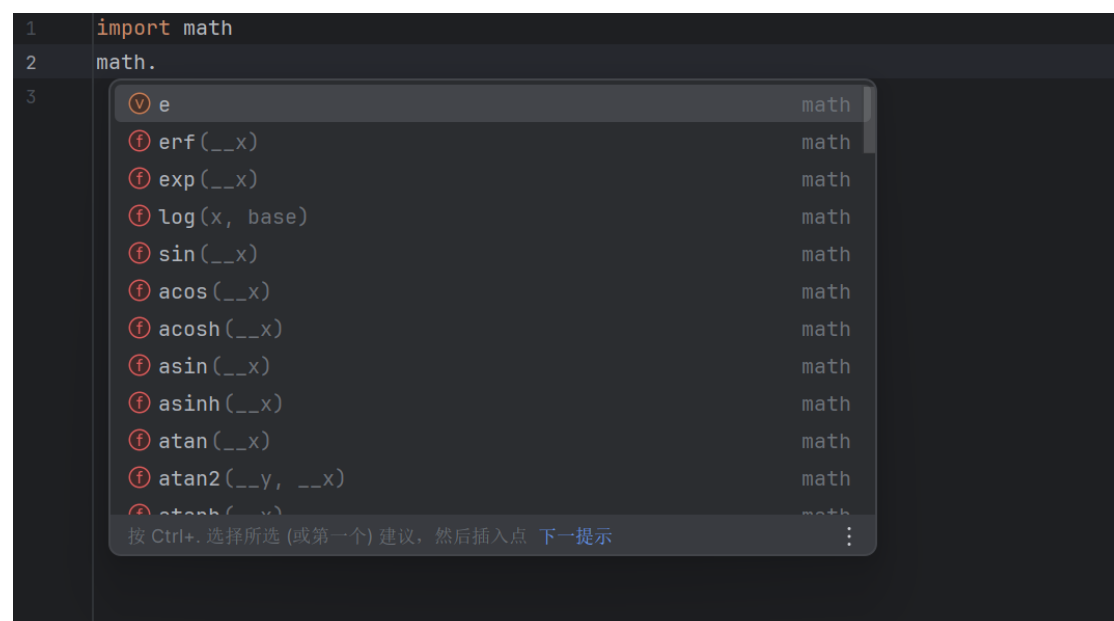
“+=” 和 “-=” 同样可用， $a+=b$ 与 $a=a+b$ 一致

```
1 print(2**3)
2
8
```

函数库 math

在 Python 中调用函数库要用 **import** 关键字，调用 math 函数后，我们就可以使用里面的很多用于数学计算的函数。

```
1 import math
2 math.
3
```



The screenshot shows a code editor with the following code:

```
1 import math
2 math.
3
```

A dropdown menu is open under the second line, showing a list of functions from the math module. The functions listed are:

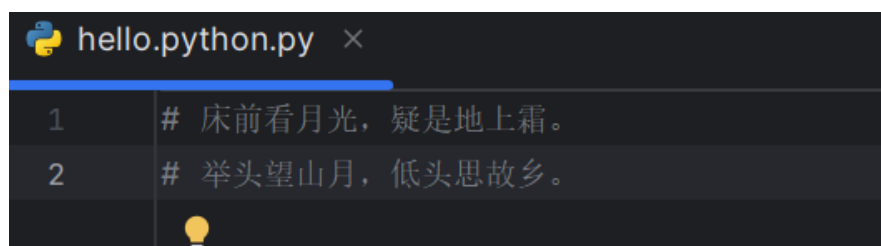
- e
- erf(__x)
- exp(__x)
- log(x, base)
- sin(__x)
- acos(__x)
- acosh(__x)
- asin(__x)
- asinh(__x)
- atan(__x)
- atan2(__y, __x)
- atanh(__x)

At the bottom of the dropdown, there is a hint: "按 Ctrl+. 选择所选 (或第一个) 建议, 然后插入点 下一提示".

3.5 注释

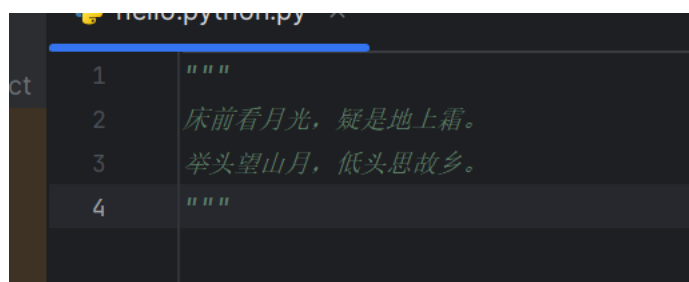
在 Python 中，注释是用来解释代码和提高代码可读性的重要工具。Python 支持两种注释方法：

1. 单行注释：使用 `#` 符号来表示单行注释。`#` 后面的内容会被解释器忽略，不会对程序产生任何影响。（快捷键：Ctrl 键 + /键）例如：



```
hello.python.py x
1 # 床前看月光，疑是地上霜。
2 # 举头望山月，低头思故乡。
```

2. 多行注释：Python 使用三个单引号 `'''` 或三个双引号 `"""` 来表示多行注释。通常用于对函数、类或模块进行注释说明。例如：



```
hello.python.py x
1 """
2 床前看月光，疑是地上霜。
3 举头望山月，低头思故乡。
4 """
```

在实际编程中，良好的注释习惯可以帮助他人更容易理解你的代码，也有助于自己在以后阅读代码时更快地理解代码的功能和目的。

3.6 数据类型

在 Python 中，常见的数据类型包括：

1. 整数 (int)：表示整数，例如 5、10、-3 等。
2. 浮点数 (float)：表示带有小数点的数值，例如 3.14、-0.001 等。
3. 字符串 (str)：表示文本数据，用单引号或双引号括起来，例如 'Hello'、"Python" 等。
4. 布尔值 (bool)：表示逻辑值，只有两个取值：True 和 False。
5. 列表(list)：表示一组有序的元素，可以包含不同类型的数据，例如 [1, 2, 3]、['apple', 'banana', 'orange']。
6. 元组 (tuple)：与列表类似，但是元组是不可变的，用小括号来表示，例如 (1, 2, 3)。
7. 集合 (set)：表示一组互不相等的元素，用大括号来表示，例如 {1, 2, 3}。
8. 字典 (dict)：表示键值对的集合，用大括号来表示，例如 {'name': 'Alice', 'age': 25}。
9. NoneType：表示空值或者空对象，用 None 来表示。

这些是 Python 中最常见的数据类型，每种数据类型都有自己的特点和用途。前四个和 C++ 是差不多的。

字符串类型 (str)

对于字符串型 str，在引号内无论是数字、字母、标点甚至空格都占有一个长度，要注意的是，一个完整的转义字符才占一个长度。我们可以通过索引获得其位于某位置的字符，代码为：变量名[n], n 是所在位置的数字，和 C++ 一样，位置从零算起。

```
1 a="Hello\nPython"
2 print(a)
3 print(a[0])
4 print(a[2])
5 print(a[3])
6 print(a[4])
7 print(a[5])
8 print(a[6])
9 print(a[7])
10 print(a[8])
11 print(a[9])
12 print(a[10])
13 print(a[11])
```

```
Hello
Python
H
l
l
o

P
y
t
```

空值类型 (NoneType)

None 表示空，不是 0 而是没有的意思，当你有变量不知道赋什么值时，可以行再试把它设为空

```
my_name=None
```

type () 函数可以返回括号内值的数据类型

```
print(type(666))
print(type('hello'))
print(type(6.6))
print(type(True))
```

```
<class 'int'>
<class 'str'>
<class 'float'>
<class 'bool'>
```

列表类型 (list)

在 Python 中，列表 (List) 是一种非常常用的数据类型，用于存储一组有序的元素。列表可以包含任意类型的元素，包括整数、浮点数、字符串、甚至其他列表等。列表使用方括号 `[]` 来表示，其中的元素之间使用逗号分隔。

下面是一个简单的列表示例：

```
1 fruits = ["apple", "banana", "cherry", "date"]
```

在这个例子中，`fruits` 是一个包含四个字符串元素的列表。列表中的元素可以通过索引来访问，索引从 0 开始。例如，要访问列表中的第一个元素，可以使用 `fruits[0]`。

除了访问元素，列表还支持添加、删除、修改元素等操作。例如，可以使用 `append()` 方法向列表末尾添加新的元素：

还可以使用 `insert()` 方法在指定位置插入新的元素：

另外，可以使用 `remove()` 方法删除指定的元素：

```
1 fruits = ["apple", "banana", "cherry", "date"]
2 #元素索引
3 print(fruits[0])
4 #末尾加元素
5 fruits.append("bear")
6 print(fruits)
7 #指定位置插入元素
8 fruits.insert( __index: 2, __object: "blueberry")
9 print (fruits)
0 #删除元素
1 fruits.remove("cherry")
2 print(fruits)
```

```
apple
['apple', 'banana', 'cherry', 'date', 'bear']
['apple', 'banana', 'blueberry', 'cherry', 'date', 'bear']
['apple', 'banana', 'blueberry', 'date', 'bear']
```

进程已结束，退出代码为 0

列表还支持切片操作，可以通过切片来获取列表中的子集。例如，`fruits[1:3]` 表示获取从第 1 个元素到第 3 个元素（不包括第 3 个元素）的子列表。

```
1 fruits = ["apple", "banana", "cherry", "date"]
2 print(fruits[1:3])
```

```
['banana', 'cherry']
```

进程已结束，退出代码为 0

列表和 C++ 中的数组有点像，但有一个很好的一点是，列表初始可以为空

```
1 fruits = []
```

列表是一种非常灵活和强大的数据类型，可以用于存储和操作各种类型的数据。列表的使用频率非常高，在实际编程中经常用于存储和处理一组相关的数据。

字典类型 (dict)

字典使用大括号 `{}` 来表示，其中的每个元素都是一个键值对（所谓键值对，就是一个键对应一个值，就像我们使用汉语字典一样，键就相当于我们要查询的那个字，值就相当于那个字的释义），键和值之间使用冒号 `:` 分隔，不同的键值对之间使用逗号 `,` 分隔。

```
person={'name':"小明",'age':19,'university':"合工大"}
```

在这个例子中，`person` 是一个包含三个键值对的字典。可以通过键来访问字典中的值

```
person={'name':"小明",'age':19,'university':"合工大"}
print(person["name"])
print(person['age'])
print(person["university"])
```

小明

19

合工大

与列表类似，字典也支持添加、删除、修改元素等操作。

```

1 person={'name':"小明",'age':19,'university':'合工大'}
2 person['age']=26
3 person['hometwon']="山东"
4 del(person['name'])
5 print(person)

```

```
{'age': 26, 'university': '合工大', 'hometwon': '山东'}
```

字典的键可以是任意不可变的数据类型，例如整数、字符串、元组等，而值可以是任意类型的数据，包括列表、字典等。

3.7 input () 函数

在 Python 中，`input()` 函数用于从用户处获取输入。它会在程序执行时暂停，并等待用户输入一些内容，然后将用户输入的内容作为字符串返回。

input() 函数的基本语法如下：

```
user_input = input("提示信息：")
```

其中，"提示信息：" 是一个可选的参数，用于向用户显示提示信息，告诉用户应该输入什么内容。用户输入的内容将作为一个字符串赋值给变量 `user\_input`。

```

a=input('你今天开心吗?')
print(a)
print(type(a))

```

```

你今天开心吗? 开心
开心
<class 'str'>

```

需要注意的是，无论我们输入的是文字还是数字，input 返回的都是字符串型，所以如果我们想使用获取的信息进行计算，需要进行数据类型的转换，转换为整型就用 int，浮点型用 float

```

1 # 计算体重指数BMI=体重 (kg) / 身高**2 (m)
2 w=input("请输入您的体重 (kg)：")
3 h=input("请输入您的身高 (m)：")
4 bmi=float(w)/float(h)**2
5 print("您的BMI指数为："+str(bmi))
6
7

```

```
请输入您的体重 (kg) : 67.5
请输入您的身高 (m) : 1.83
您的BMI指数为: 20.155872077398545
```

3.8 if 条件语句

Python 的 if 语句和 C++ 只有书写格式不同。C++ 判断条件要括起来，执行语句一般也括起来封装。在 Python 中，if 的判断条件不用括，但是后面要加冒号，执行语句用缩进（一般为 4 个空格）表明所属。格式为

if 判断条件:

 执行语句 1

 执行语句 2

.....

else:

 执行语句 1

 执行语句 2

.....

Python 中的条件关系判断符与 C++ 一致

等于== 不等于!= 大于> 小于< 小于等于<= 大于等于>=

逻辑运算符不一样

真: and 或: or 非: not

```
1  # 计算体重指数BMI=体重 (kg) / 身高**2 (m)
2  w=input("请输入您的体重 (kg) : ")
3  h=input("请输入您的身高 (m) : ")
4  bmi=float(w)/float(h)**2
5  print("您的BMI指数为: "+str(bmi))
6  if bmi>=18.5 and bmi<=23.9:
7      print("您的体重指数在正常范围内")
8  else:
9      print("您的身高体重比例不好，不健康")
10
```

```
请输入您的体重 (kg) : 67.5
请输入您的身高 (m) : 1.83
您的BMI指数为: 20.155872077398545
您的体重指数在正常范围内

进程已结束，退出代码为 0
```

和 C++ 很不一样的一点是，上面代码中的 “`bmi>=18.5 and bmi<=23.9`”，在 Python 中可以连起来写

```
1 w=input("请输入您的体重 (kg) :")
2 h=input("请输入您的身高 (m) :")
3 bmi=float(w)/float(h)**2
4 print("您的BMI指数为: "+str(bmi))
5 if 18.5<=bmi<=23.9:
6     print("您的体重指数在正常范围内")
7 else:
8     print("您的体重指数不在正常范围内")
9
```

if 的嵌套语句（多条件 if 语句）

if 的嵌套语句同样用缩进体现

```
1 mood=float(input("女朋友今天的心情指数: "))
2 if mood<=6:
3     if int(input("女朋友在不在家 (1.在 2.不在): "))==2:
4         print("可以打游戏")
5     else:
6         print("别打游戏")
7 else:
8     print('可以打游戏')
9
10
11
```

```
女朋友今天的心情指数: 1
女朋友在不在家 (1.在 2.不在): 2
可以打游戏
```

在 Python 中 else if 语句的符号变为了 `elif`

```

1 w=input("请输入您的体重 (kg) :")
2 h=input("请输入您的身高 (m) :")
3 bmi=float(w)/float(h)**2
4 print("您的BMI指数为: "+str(bmi))
5 if 18.5<=bmi<=23.9:
6     print("您的体重指数在正常范围内")
7 elif bmi<18.5:
8     print("您有点瘦")
9 else:
10    print("您有点胖")
1

```

```

请输入您的体重 (kg) :50
请输入您的身高 (m) :2.0
您的BMI指数为: 12.5
您有点瘦

```

3.9 for 循环

在 Python 中，for 循环用于遍历序列（例如列表、元组、字典等）或者其他可迭代对象中的元素。for 循环的基本语法如下：

```

for 变量 in 可迭代对象:
    # 执行语句

```

在这个语法中，“变量”表示在每次循环中被赋值的变量，“可迭代对象”是一个包含多个元素的对象，例如列表、元组、字典的键、字符串等。

遍历列表

```

1 temperature_list=[37,36.5,37.5,39,36.7,37.8]
2 for temperture in temperature_list:
3     if temperture>37.3:
4         print(str(temperture)+' 您发烧了')

```

37.5 您发烧了

39 您发烧了

37.8 您发烧了

遍历字典

```

1 person = {"name": "Alice", "age": 25}
2 for key in person:
3     print(key, person[key])

```

```

name Alice
age 25

```

此外，Python 还提供了内置函数 `range()`，可以用于生成一系列数字，结合 `for` 循环可以进行一定范围的遍历

```

1 total=0;
2 for i in range(1,101):#第一个数代表起始值，第二个代表终止值，但不被包含在内
3     total=total+i
4     print(total)
5
6 for i in range(1,10,2):#这里的第三个值代表步长，不写时默认为1
7     print(i)

```

```
5050
```

```

1
3
5
7
9

```

总之，`for` 循环是 Python 中非常常用的循环结构，可以用于遍历各种类型的数据结构，进行重复操作。

3.10 while 循环

Python 中 `while` 的基本语法如下

While 判断语句：

执行语句

```

1 temperature=[37,36.5,37.4,38,38.2,36.5]
2 i=0
3 while i<len(temperature):
4     if(temperature[i]>37.3):
5         print(str(temperature[i])+"您发烧了")
6     i += 1

```

```
37.4您发烧了
```

```
38您发烧了
```

```
38.2您发烧了
```

3.11 格式化字符串

如果我们想把下面贺词的“虎”改为任意一个生肖年

虎年到来鞭炮闹，红妆对联也娇俏。

迎春花 儿悄然笑，祝福声声不停。

愿你在虎年伊始之际把幸福紧紧套牢，把快乐紧紧拥抱，和好运互相依靠，把忧伤统统抛掉
虎年快乐。

现有知识是这样做

```
year='虎'
print(
    year+'''年来鞭炮闹，红妆对联也娇俏。
    迎春花 儿悄然笑，祝福声声不停。
    愿你在''' + year + '''年伊始之际把幸福紧紧套牢，把快乐紧紧拥抱，和好运互相依靠，把忧伤统统抛掉。'''
    + year + '年快乐。'
)
```

这样显得很乱

格式化字符串就可以让我们的代码看起来很规整

format 方法

```
1 information='''{0}年来鞭炮闹，红妆对联也娇俏。
2     迎春花 儿悄然笑，祝福声声不停。
3     愿{1}在{0}年伊始之际把幸福紧紧套牢，把快乐紧紧拥抱，和好运互相依靠，把忧伤统统抛掉，{0}年快乐。
4     '''.format(*args: '虎', '你')
5 print(information)]
```

```
虎年到来鞭炮闹，红妆对联也娇俏。
迎春花 儿悄然笑，祝福声声不停。
愿你在虎年伊始之际把幸福紧紧套牢，把快乐紧紧拥抱，和好运互相依靠，把忧伤统统抛掉，虎年快乐。
```

这里的{x}是位置索引，可以把后面 format 里面相应位置的数填充进去

还可以这样

```
1 information='''{year}年来鞭炮闹，红妆对联也娇俏。
2     迎春花 儿悄然笑，祝福声声不停。
3     愿{person}在{year}年伊始之际把幸福紧紧套牢，把快乐紧紧拥抱，和好运互相依靠，把忧伤统统抛掉，{year}年快乐。
4     '''.format(person='你', year='虎')
5 print(information)]
```

这样就无所谓先后顺序了

f-字符串

```
1 year='虎'
2 person='你'
3 information=f'''{year}年来鞭炮闹，红妆对联也娇俏。
4     迎春花 儿悄然笑，祝福声声不停。
5     愿{person}在{year}年伊始之际把幸福紧紧套牢，把快乐紧紧拥抱，和好运互相依靠，把忧伤统统抛掉，{year}年快乐。
6     '''
7 print(information)]
```

此时花括号内的变量会被直接求值替换

3.12 函数

Python 中设定函数的关键字是“def”，所谓函数就用来防止一段代码当需要重复使用时，就将这段代码写到一个函数里，使用时就调用这个函数，防止代码重复太多

基本语法

def 函数名 () :

 函数执行语句

 return (返回值)

```
#求两个整数的最大公因数
1个用法
def great_common_factor(m,n):
    if m<n:
        temp=n
        n=m
        m=temp
    while m%n!=0:
        i=m%n
        m=n
        n=i
    return n
```

3.13 引入函数库

在之前的 3.4 数学运算中，我们就引入了函数库 math。现在综合讲讲“引入函数库”

1. import 模块名 （引入整个模块）

```
1 import math
2 math.sqrt(4)#调用开方函数
```

2. from 模块名 import 函数名 （引入库中的特定函数）

```

1  from math import sqrt,sin
2  sqrt(4)
3  sin(0)

```

可以看出，使用这种方法是函数直接用，不需要“math.”

3. from 模块名 import * （引入库中的所有函数）

```

1  from math import *
2  sqrt(4)
3  sin(0)

```

此方法同样可以直接调用函数，但不推荐使用，首先这个方法可能会引入很多不需要的函数，更重要的是，如果你引入两个库且其中有同名函数，调用这个函数是系统会报错。

查找 Python 标准库

打开 Python 官网，在“文档”下面找到“Docs”，在找到里面的“库参考”即可

Python 是一种编程语言，可

Python's standard documentation: download, browse or watch a tutorial.

Get started below, or visit the Documentation page to browse by version.

Python Docs

Python 3.12.1 文档 - (模块索引)

- 浏览 Python 3.12.1 中的新增功能
- 教程
- 库参考
- 语言参考
- 扩展和嵌入
- Python/C 接口
- 使用 Python
- Python 贴士指南
- 词汇表
- 搜索在线文档
- 下载当前文档 (提供多种格式，包括用于打印的排版。)

从 Python 2 移植到 Python 3

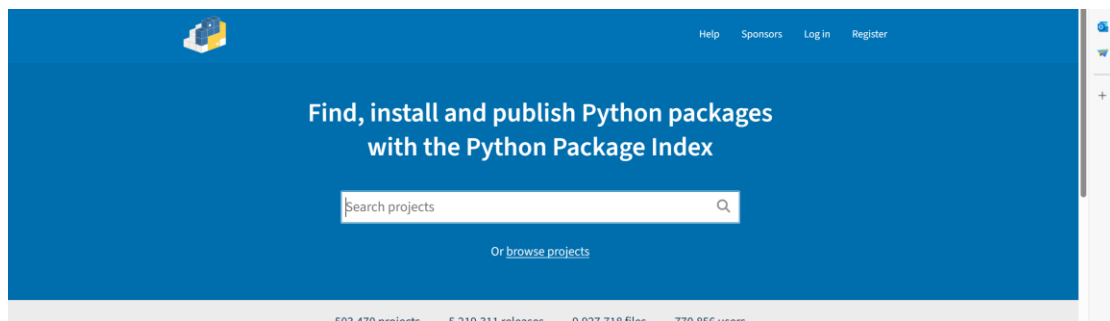
- 常见问题解答: 停用 Python 2
- Python 2.7 最终发布时间表
- Python 3 语句
- 将 Python 2 代码移植到 Python 3
 - 确定哪些项目阻止您移植到 Python 3
 - Python 2 支持和迁移



有些函数可能标准库里没有，此时我们可以查找第三方库，就是其他程序员写的非官方的库。

查找网站：[PyPI · The Python Package Index](https://pypi.org/)

搜索（英文）你需要的库



然后打开终端输入：`pip install 库的名字`

下载库，之后就可以引用了



4 面向对象编程

4.1 类

在 Python 中，类是一种用于创建对象的蓝图或模板。类定义了对象的属性（称为属性）和行为（称为方法）。当你创建一个类的实例时，你创建了一个特定类型的对象，该对象包含了类中定义的属性和方法。

“属性”即该类对象具有的一些性质，这个通常用类的构造函数来操作

```
2 个用法
1  class Person:
2     def __init__(self, name, sex, age):
3         self.name = name
4         self.sex = sex
5         self.age = age
6
7     2 个用法
8     def run(self):
9         print(f'{self.age}岁的{self.sex}同学{self.name}正在跑步')
10
11 person01 = Person(name='小明', sex='男', age=19)
12 person02 = Person(name='小红', sex='女', age=10)
13 person01.run()
14 person02.run()
```

19岁的男同学小明正在跑步

10岁的女同学小红正在跑步

进程已结束，退出代码为 0

构造函数“__init__”必须是“双下划线+init+双下划线”，然后在在类中无论是构造函数还是别的函数（方法），第一个参数都只能是 self，用于指向自身属性，类似于 C++ 中的 this 指针

4.2 继承

“继承”就是把一个类的属性和方法给另一个类，继承的那个类叫子类，被继承的叫父类

语法是

```
class 子类名(父类名):
```

如果子类没有构造函数，将会执行父类的构造函数，有构造函数就执行自己的，如果我们还想获得父类构造函数中的属性，可以用 super()，它可以返回当前类的父类，把父类的构造函数调用一下就行

如果子类有和父类重名的方法，在调用时会调用子类的方法。

```
2 个用法
1 class Animal:
2     def __init__(self, name, sex, age):
3         self.name = name
4         self.sex = sex
5         self.age = age
1 个用法
6     def run(self):
7         print(f'{self.age}岁的{self.sex}性动物{self.name}正在奔跑')
1 个用法
8 class Person(Animal):
9     def run(self):
10        print(f'{self.age}岁的{self.sex}同学{self.name}正在跑步')
1 个用法
11 class Cat(Animal):
12     def __init__(self, name, sex, age):
13         super().__init__(name, sex, age)
14         self.has_tail = False
15 person01 = Person(name='张三', sex='男', age=33)
16 cat01 = Cat(name='糯米', sex='雌', age=5)
17 person01.run()
18 cat01.run()
```

```
33岁的男同学张三正在跑步
5岁的雌性动物糯米正在奔跑
```

5. 程序处理

5.1 异常处理

当代码出错时，出错代码后面的所有代码都无法运行，很多时候这不是我们想要的，此时我们可以用“try”关键字来捕捉错误。

用法是用 try 将可能出现错误的代码包含，然后用 except 加上错误类型来捕捉错误

```

1  try:
2      w = input("请输入您的体重 (kg) :")
3      h = input('请输入您的身高 (m) :')
4      bmi = float(w) / float(h) ** 2
5  except ValueError:
6      print('输入不合理数字, 请重新运行程序, 输入合理数字')
7  except ZeroDivisionError:
8      print('身高不能为零, 请重新运行程序, 输入合理身高')
9  except:
10     print('发生未知错误, 请重新运行程序')
11  else:
12     print('您的BMI指数是: ' + str(bmi))
13  finally:
14     print('程序运行结束')

```

当我们输入身高体重时, 如果输入的是非数字, 程序就会直接崩掉, 因为它无法转化为数字也无法运算, 此时发生的错误就是 `ValueError`, 所以我们用 `except ValueError` 来捕捉这个错误

```

C:\Users\杜晓川\PycharmProjects\pythonProject\.venv\Scripts\python
请输入您的体重 (kg) : 一行白鹭上青天
请输入您的身高 (m) : 两只黄鹂鸣翠柳
输入不合理数字, 请重新运行程序, 输入合理数字
程序运行结束

进程已结束, 退出代码为 0

```

在 BMI 的计算公式中身高作为除数, 是不能等于零的, 如果等于零, 就会发生除零错误, 也就是 `ZeroDivisionError`, 所以我们用 `except ZeroDivisionError` 来捕捉错误

```

C:\Users\杜晓川\PycharmProjects\pythonProject\.venv\Scripts\python
请输入您的体重 (kg) : 67
请输入您的身高 (m) : 0
身高不能为零, 请重新运行程序, 输入合理身高
程序运行结束

进程已结束, 退出代码为 0

```

当然如果你不知道错误类型, 可以用 `except` 来接收所有错误。

需要注意的是, 当有一个错误被接受到后, 其他的接受错误的代码都不会运行

在最后, 我们还可以用 `finally` 关键字, 这里面的代码无论是程序运行成功或失败都会被执行

```
请输入您的体重 (kg) :67.5
请输入您的身高 (m) :1.83
您的BMI指数是: 20.155872077398545
程序运行结束

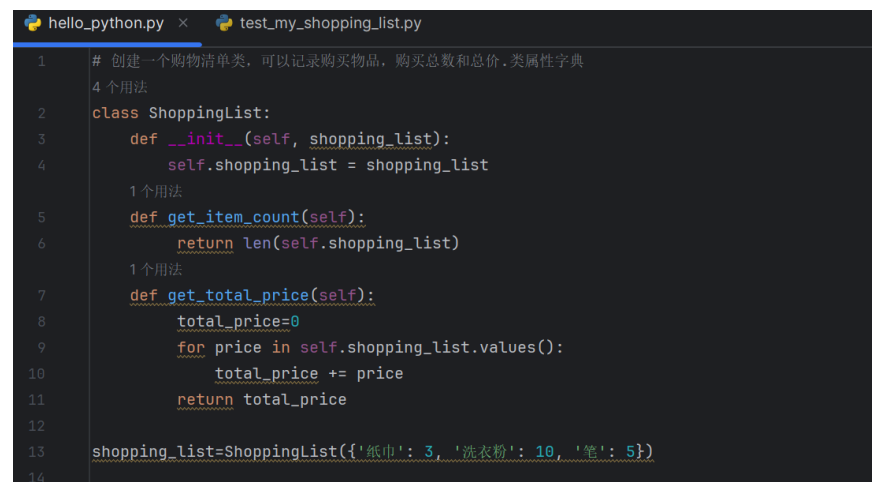
进程已结束, 退出代码为 0
```

5.2 程序测试

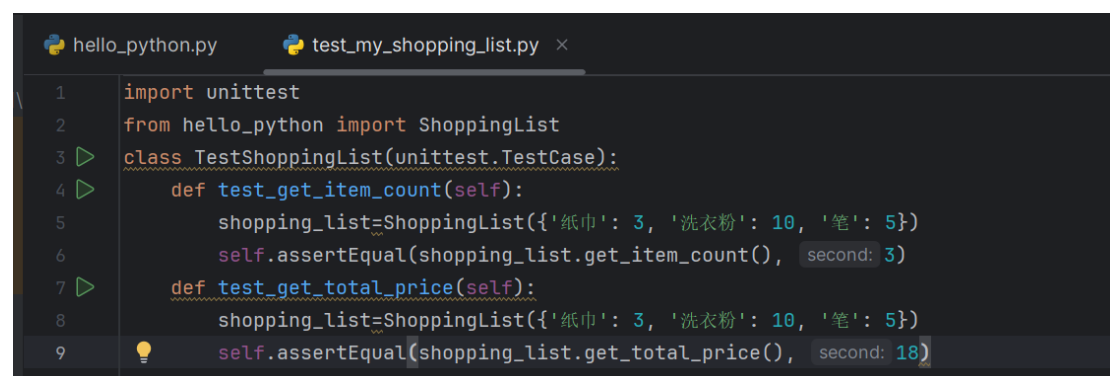
这节其实就是教我们如何测试程序，找 bug

首先我们要了解 Python 单元测试库，它可以帮助我们测试程序，找错误，这里讲一个 Python 自带且常用的单元测试库 unittest

首先我们先在创建一个文件，专门用于程序测试，这样将实现代码和测试代码分开写，更清晰明了



```
hello_python.py × test_my_shopping_list.py
1 # 创建一个购物清单类，可以记录购买物品，购买总数和总价，类属性字典
2 4 个用法
3 class ShoppingList:
4     def __init__(self, shopping_list):
5         self.shopping_list = shopping_list
6     1 个用法
7     def get_item_count(self):
8         return len(self.shopping_list)
9     1 个用法
10    def get_total_price(self):
11        total_price=0
12        for price in self.shopping_list.values():
13            total_price += price
14        return total_price
15
16 shopping_list=ShoppingList({'纸巾': 3, '洗衣粉': 10, '笔': 5})
```



```
hello_python.py test_my_shopping_list.py ×
1 import unittest
2 from hello_python import ShoppingList
3 class TestShoppingList(unittest.TestCase):
4     def test_get_item_count(self):
5         shopping_list=ShoppingList({'纸巾': 3, '洗衣粉': 10, '笔': 5})
6         self.assertEqual(shopping_list.get_item_count(), second: 3)
7     def test_get_total_price(self):
8         shopping_list=ShoppingList({'纸巾': 3, '洗衣粉': 10, '笔': 5})
9         self.assertEqual(shopping_list.get_total_price(), second: 18)
```

我们在第一个文件里创建一个类，之后在第二个文件里测试这个类

首先要引入 `unittest` 库，然后再引入我们写的类，格式是 `from 文件名（注意文件名的格式一定要对，不然可能报错） import 类名或函数名`

然后我们创建一个测试类，将 `unittest.TestCase` 作为它的父类，这样就可以使用该类的所有测试方法了

然后我们创建测试的方法，名称必须以“`test_`”开头，系统会自动将这样的方法当作测试运行

`unittest.TestCase` 类中有很多测试方法，我们把需要比较的东西传入即可

unittest.TestCase类的常见测试方法	
方法	类似于
<code>assertEqual(A, B)</code>	<code>assert A == B</code>
<code>assertTrue(A)</code>	<code>assert A is True</code>
<code>assertIn(A, B)</code>	<code>assert A in B</code>
<code>assertNotEqual(A, B)</code>	<code>assert A != B</code>
<code>assertFalse(A)</code>	<code>assert A is False</code>
<code>assertNotIn(A, B)</code>	<code>assert A not in B</code>
类似于 <code>assert 参数1 != 参数2</code>	

比如第一个 `assertEqual(A, B)`，我们把两个可以比较的值传入进去，如果相等，则测试成功，否则显示失败

```
Testing started at 20:48 ...

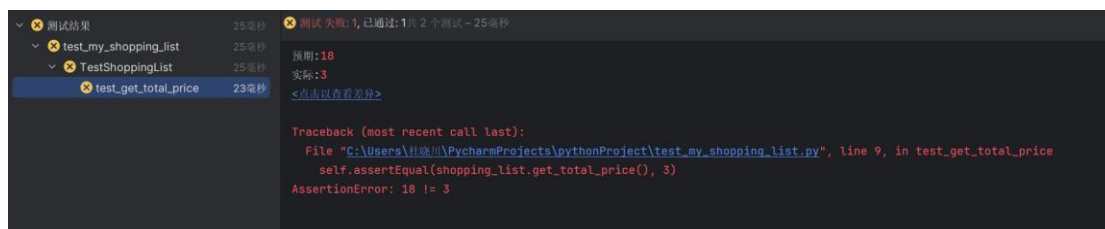
Ran 2 tests in 0.003s

OK

Launching unittests with arguments python -m unittest C:\Users\杜晓川\PycharmProjects\p
```

这是代码运行结果，ok 说明没问题。

如果我们把上面 18 改为 3



左边会给我们指出错误地方，右边框会指出错因

6. 写在最后

6.1 心得体会

花费几天的时间，也算是对 Python 入了一点门，同时编写了一篇粗略的学习教程，挺有感触的。

首先是对编程语言的学习这块。已经有了学习 C++ 的基础，再去学习一门新的语言真的感觉很简单，很容易理解，两门语言有很多相似的甚至可以说是重合的内容。Python 确实比 C++ 要便捷很多，代码简洁且更直接，像列表、字典这些数据类型使用起来真的很方便。还有那么多的库可以调用。像专门测试、找错的这些东西真的很人性化，在 C++ 里应该没有吧。当然从一门语言换到另一门语言刚开始写代码时总会有点不适应，比如我在写完一句代码时就习惯打上分号，还有 Python 的缩进规则真的搞不懂，我明明感觉自己缩进的没有问题，很清晰直观，但有时候系统就是给我一堆警告，说我违反了 PEP8，这点很烦。

然后是自己编写一份入门级别的学习教程。说是自己编写，其实不过是将我学习的别人的视频课程转化成文字和图片罢了，再在网上找找一些概念讲解什么的（我一般是直接问 AI）。有时候还要想怎么让自己的语言更加简练，讲解更加通俗易懂，还是有点难度的。编写教程的过程很繁琐，远远超过我学习 Python 的时间，但最终看到我这几十页，几千字的教程终于完成时，还是挺有成就感的。

6.2 参考材料

什么是 Python

[Python 语言介绍 - 知乎 \(zhihu.com\)](https://www.zhihu.com/question/20462016)

为什么安装 PyCharm

[PyCharm 为什么这么牛？ - 知乎 \(zhihu.com\)](https://www.zhihu.com/question/20462016)

学习课程

https://www.bilibili.com/video/BV1944y1x7SW/?spm_id_from=333.337.search-card.all.click

ChatGPT